in doing that much coverage work at once are high, especially if the customer is expecting you to start working on new functionality.

You draw a line in the sand and start working in a test-driven mode moving forward. One critical element is to ensure that every bug fix starts by writing a failing test somewhere—whether unit, functional, or integration. This is a good way to ramp up tests on your project and it allows you to organically build test coverage over time with relatively small risk to your deadlines and fairly little chance of breaking existing functionality.

Similarly, new features must be added using a TDD process. In the beginning this often requires the heightened use of mock objects, but over time the codebase and the test coverage both improve.

If you're like me, the temptation to clean up the entire codebase at once can be almost overwhelming. In this situation, lie down until the feeling passes or you're so close to your deadline that fixing everything is no longer a viable option.

Finally, *do one thing at a time, to the extent possible.* Don't extend test coverage while you're adding new functionality. And don't try to clean up the code while you're extending test coverage (occasionally this will be unavoidable, but keep it to a minimum). The fewer things you have moving at any one time, the easier it will be to identify the culprit when things go wrong.

What You've Done

In this chapter you explored a couple of techniques to help you when you're trying to test a legacy Rails project. You saw how integration tests can be used to create test coverage without being concerned with the structure of the legacy code. You then saw how to use unit tests to build an understanding of how the code works. You also explored techniques to inject code into the system to change its behavior during test so as to focus the test on a particular piece of code.

Over the entire book, you've seen how to use automated developer testing to improve your Rails application, making it easier to validate behavior, and how to use tests to create good code that is easy to change. I've always found writing tests to be fun, and making tests pass is a great way to give myself small challenges along the way to understanding and writing code. I hope that you can use these techniques to save yourself time, make your coding less stressful, and write code that has value.